InsightFlow Architecture Specification

Technical Overview, Data Ingestion, Processing Pipeline, and Data Model

1. System Architecture Overview

InsightFlow is a distributed, event-driven web platform designed to ingest, process, and visualize customer feedback at scale. The architecture decouples the user-facing web server from heavy I/O-bound web scraping and latency-heavy AI inference tasks.

- **Frontend Client:** Next.js (React) provides a responsive, functional web application with a polished UI/UX.
- **API Gateway:** FastAPI (Python) handles rapid HTTP requests, schema validation, and database querying.
- **Message Broker:** Redis queues background tasks and acts as a state tracker.
- **Async Workers:** Celery (or RQ) Python processes run independently of the API to execute headless browsers (Playwright) and agentic workflows (LangGraph).
- **Intelligence Engine:** The Adaption Labs API is integrated to process text contextually, satisfying the requirement that Adaption must contribute to a meaningful part of the AI or data workflow.
- **Relational Storage:** Supabase (PostgreSQL) handles complex table joins for the analytics metrics.

2. Ingestion & Data Collection Layer

To fulfill the requirement to handle multiple sources of feedback (including reviews, surveys, and support messages), the system routes data through three distinct entry points.

- **Synchronous Import (`POST /api/ingest/csv`):** A standard FastAPI route utilizing the `python-multipart` library. Users can upload a file containing sample data for demonstration. The server streams the file, validates the headers against a Pydantic schema, and queues each row to Redis in batches.

- **Real-Time Webhooks (`POST /api/ingest/webhook`):** A publicly exposed endpoint designed to receive JSON payloads pushed directly from external B2B platforms like Zendesk or Typeform. It uses HMAC signature verification for authentication, normalizes the payload, and drops it into the background processing queue.
- **Asynchronous Scraper Workers:** When the frontend requests live data from an external platform, FastAPI registers a `job_id` in Redis and immediately returns an HTTP `202 Accepted`. A background Celery worker picks up the job and executes the extraction:
    - **Native API Fallbacks:** If targeting an open-source bug tracker, the worker fires a `requests` call to the GitHub REST API to fetch recent issues.
    - **Agentic Extraction (LangGraph + Groq):** If targeting a complex web page (e.g., Trustpilot), the worker launches an invisible Playwright Chromium instance to bypass static blocks and capture the raw HTML. LangGraph passes this HTML into a fast-inference LLM (Groq or Mistral) prompting the AI to locate and extract only the raw customer comments.

3. Data Normalization & Processing Pipeline

Data retrieved from the ingestion layer is highly irregular. Before insertion into the database, it must pass through a two-stage structural and semantic pipeline.

Stage 1: Deterministic Structural Cleaning (FastAPI / Pydantic)

Every piece of data must map to the BaseFeedbackInput Pydantic model (id, source, raw_text, timestamp, rating). The backend drops completely empty rows, deduplicates inputs by ID, and casts all dates to ISO 8601 strings so the frontend can accurately visualize trends and changes in feedback. Basic Regex sanitization scrubs explicit Personally Identifiable Information (PII) like phone numbers.

Stage 2: Intelligence & Semantic Shaping (Adaption API)

The Celery worker passes the structurally cleaned text payloads strictly to the Adaption Labs API. This stage is responsible for all heavy cognitive tasks required to analyze feedback to identify sentiment, recurring topics, and common complaints.

- **Contextual Shaping:** Adaption detects keyboard spam, corrects severe typographical errors, and normalizes slang without destroying the core sentiment.
- **Categorization:** Adaption maps the raw text into the business's specific custom taxonomy, actively grouping similar feedback into meaningful categories.
- **Priority Extraction:** Adaption evaluates aggregated complaints to automatically identify high-priority issues based on feedback patterns. It generates a JSON object containing actionable recommendations supported by the analyzed data, and outputs a natural language string to explain why particular issues deserve attention.

4. Database Schema & State Management (PostgreSQL)

The processed data is written to normalized PostgreSQL tables designed for rapid analytical queries.

Table Name	Column Name	Data Type	Key / Constraints	Description
`feedback_entries`	`id`	UUID	Primary Key	Unique identifier for raw feedback entry
	`source_platform`	String		Origin platform (e.g., "Zendesk", "CSV")
	`timestamp`	Timestamptz		Event timestamp normalized to ISO 8601
	`raw_text`	Text		Original cleaned text content
	`customer_rating`	Integer	Nullable	Optional user score (1–5)
`ai_analyses`	`feedback_id`	UUID	Foreign Key → `feedback_entries.id`	Associated raw feedback record
	`sentiment_label`	Enum	'Positive', 'Neutral', 'Negative'	Inferred sentiment classification

Table Name	Column Name	Data Type	Key / Constraints	Description
	`sentiment_score`	Float		Confidence score ranging [-1.0, 1.0]
	`primary_category`	String		Classified taxometric bucket
`priority_issues`	`issue_id`	UUID	Primary Key	High-level aggregated issue tracking
	`title`	String		Short descriptive title of the issue
	`priority_level`	Enum	'Critical', 'High', 'Low'	AI-assessed urgency rating
	`ai_explanation`	Text		Justification for flagged issue
	`ai_recommendation`	Text		Proposed resolution step
	`status`	Enum	'Open', 'Tracked', 'Resolved'	Current workflow state

5. Analytics Engine & UI Visualization

The Next.js frontend interfaces with FastAPI via highly specialized aggregation endpoints designed to present the findings in a clear, business-friendly format.

- **`GET /api/metrics/dashboard`:** Executes SQL `GROUP BY` operations on the `ai_analyses` table to calculate total feedback volume and the average sentiment distribution. The frontend uses this payload to render top-row KPI cards and a primary Donut Chart.
- **`GET /api/metrics/categories`:** Returns a frequency count of the taxonomy generated by Adaption. The frontend visualizes this via a Horizontal Bar Chart to highlight frequently reported issues.
- **`GET /api/metrics/trends`:** Groups feedback volume and sentiment metrics by chronological intervals (days or weeks) to populate a Time-Series Line Chart.

- **Issue Tracking Interface:** An interactive section querying the `priority_issues` table. Users can read the AI's justification for the alert, view the suggested action plan, and update the row's `status` to save or track important issues.
- **Interactive Data Grid:** A paginated table querying `feedback_entries` joined with `ai_analyses`. It implements dynamic dropdown states allowing users to actively filter results by category, sentiment, or other relevant attributes.
- **Report Generation (`GET /api/export`):** A backend controller that serializes the currently filtered dashboard view into a downloadable document, allowing users to export or share an insight report directly from the UI.